

# Software/Firmware: Comunicación Serial (UART)

---

## Desarrollo de comunicación UART para carga y monitoreo — Tarjeta Pinguino (PIC18F2550/4550)

Junio 2026

**Tarea:** Desarrollar comunicación UART para carga o monitoreo **Plataforma:** Tarjeta Pinguino — Microcontrolador PIC18F2550 / PIC18F4550

### 1. Descripción general

---

Esta librería implementa comunicación serial UART **bidireccional** para el PIC18F2550/4550 usando el módulo EUSART integrado del microcontrolador, a una velocidad de **9600 baudios** (8 bits de datos, sin paridad, 1 bit de stop — configuración 8N1).

La transmisión se realiza por **consulta (polling)**, esperando a que el hardware indique que el registro de transmisión está libre. La recepción se realiza por **interrupción**, guardando cada byte recibido en un **buffer circular (FIFO)** de 64 posiciones. Esto permite que el programa principal siga ejecutando otras tareas (leer sensores, controlar salidas, etc.) sin perder datos que lleguen por el puerto serial — ideal tanto para **monitoreo** (enviar lecturas a una PC) como para **carga** (recibir comandos o datos desde una PC).

La librería se compone de tres archivos:

| Archivo             | Contenido                                               |
|---------------------|---------------------------------------------------------|
| <code>uart.h</code> | Declaraciones, configuración y prototipos               |
| <code>uart.c</code> | Implementación del driver UART y buffer circular        |
| <code>main.c</code> | Ejemplo de uso: monitoreo periódico + carga de comandos |

**Pines físicos utilizados** (fijos por hardware del módulo EUSART):

| Señal | Pin del PIC |
|-------|-------------|
| TX    | RC6         |
| RX    | RC7         |

## 2. Funciones implementadas

| Función                                       | Descripción                                                                              |
|-----------------------------------------------|------------------------------------------------------------------------------------------|
| <code>uart_init()</code>                      | Configura el módulo EUSART a 9600 baudios y habilita la interrupción de recepción        |
| <code>uart_write(dato)</code>                 | Envía un byte (modo consulta/bloqueante)                                                 |
| <code>uart_print(cadena)</code>               | Envía una cadena de texto terminada en <code>'\0'</code>                                 |
| <code>uart_println(cadena)</code>             | Envía una cadena y agrega salto de línea <code>\r\n</code>                               |
| <code>uart_print_int(numero)</code>           | Convierte un entero a texto decimal y lo envía (para monitoreo)                          |
| <code>uart_available()</code>                 | Indica si hay datos disponibles para leer (1 = sí, 0 = no)                               |
| <code>uart_read()</code>                      | Lee y retira un byte del buffer de recepción                                             |
| <code>uart_read_line(destino, tam_max)</code> | Arma una línea completa de texto recibida (hasta <code>'\n'</code> ), útil para comandos |
| <code>uart_flush()</code>                     | Vacía el buffer circular de recepción                                                    |
| <code>uart_isr()</code>                       | Rutina a llamar desde la interrupción de alta prioridad                                  |

## 3. Funcionamiento del buffer circular

La recepción no bloquea el programa principal: cada vez que llega un byte por UART, ocurre una interrupción de hardware (RCIF) y `uart_isr()` guarda ese byte en `rx_buffer[]`. Dos índices (`rx_head` y `rx_tail`) llevan el control de escritura y lectura:

- `rx_head` avanza cada vez que la interrupción guarda un byte nuevo.
- `rx_tail` avanza cada vez que el programa principal lee un byte con `uart_read()`.
- Mientras `rx_head != rx_tail`, hay datos pendientes por leer (`uart_available()` devuelve 1).

Este patrón productor-consumidor evita perder bytes mientras el programa principal está ocupado en otra tarea, y es la base para implementar tanto monitoreo (enviar datos continuamente) como carga (recibir comandos completos con `uart_read_line()`).

## 4. Código fuente

---

### 4.1 uart.h

```

/* =====
 * uart.h
 * Librería de comunicación serial UART
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 *
 * Comunicación bidireccional a 9600 baudios.
 * Transmisión por consulta (polling) y recepción por
 * interrupción, usando un buffer circular (FIFO) para no
 * perder bytes mientras el programa principal hace otras
 * tareas (carga de datos / monitoreo).
 *
 * Pines físicos usados (módulo EUSART del PIC18F2550/4550):
 *   TX -> RC6
 *   RX -> RC7
 *
 * Autor: (tu nombre)
 * Tarea: Software/Firmware - Comunicación serial
 * ===== */

#ifndef UART_H
#define UART_H

#include <pl8cxxx.h>

/* ----- Configuración general ----- */
#define _XTAL_FREQ      4800000UL    // Frecuencia del oscilador (ajusta a tu
configuración)
#define UART_BAUDRATE   9600UL      // Velocidad de comunicación

/* Tamaño del buffer circular de recepción.
 * Debe ser potencia de 2 para que el operador módulo
 * funcione como máscara rápida (& (UART_RX_BUFFER_SIZE - 1)) */
#define UART_RX_BUFFER_SIZE  64

/* ----- Prototipos de funciones públicas ----- */

/* Inicializa el módulo EUSART: configura TRIS, SPBRG,
 * habilita TX/RX y la interrupción de recepción. */
void uart_init(void);

/* ----- Transmisión (modo consulta / polling) ----- */

/* Envía un solo byte/caracter por UART (función bloqueante:
 * espera a que el registro de transmisión esté libre) */
void uart_write(unsigned char dato);

/* Envía una cadena de texto terminada en '\0' */
void uart_print(const char *cadena);

/* Envía una cadena de texto seguida de salto de línea (\r\n) */
void uart_println(const char *cadena);

/* Envía un número entero (con signo) como texto en formato decimal.
 * Útil para monitoreo de variables/sensores por terminal serial. */
void uart_print_int(int numero);

```

```

/* ----- Recepción (modo interrupción + buffer circular) ----- */

/* Indica si hay al menos un byte disponible para leer
 * en el buffer de recepción. Regresa 1 (sí) o 0 (no). */
unsigned char uart_available(void);

/* Lee y retira un byte del buffer de recepción.
 * Debe llamarse después de comprobar uart_available().
 * Si el buffer está vacío regresa 0. */
unsigned char uart_read(void);

/* Lee una línea completa de texto desde el buffer (hasta
 * encontrar '\n' o llenar el arreglo destino), usada para
 * recibir comandos completos por monitor serial.
 * Regresa 1 si se completó una línea, 0 si aún no hay
 * un '\n' en el buffer disponible. */
unsigned char uart_read_line(char *destino, unsigned char tam_max);

/* Vacía por completo el buffer circular de recepción. */
void uart_flush(void);

/* ----- Rutina de interrupción -----
 * Debe llamarse desde la interrupción de alta prioridad
 * (high_isr) del programa principal, dentro del bloque
 * "#pragma interrupt" correspondiente. Ver ejemplo en main.c */
void uart_isr(void);

#endif /* UART_H */

```

## 4.2 uart.c

```

/* =====
 * uart.c
 * Implementación de la librería de comunicación serial UART
 * Tarjeta: Pinguino (PIC18F2550 / PIC18F4550)
 * ===== */

#include "uart.h"

/* -----
 * Buffer circular (FIFO) de recepción.
 * - cabeza (head): posición donde la ISR escribe el próximo byte
 * - cola (tail): posición donde uart_read() lee el próximo byte
 * Mientras head != tail, hay datos disponibles para leer.
 * ----- */
static volatile unsigned char rx_buffer[UART_RX_BUFFER_SIZE];
static volatile unsigned char rx_head = 0;
static volatile unsigned char rx_tail = 0;

/* -----
 * uart_init()
 * Configura el módulo EUSART del PIC18F2550/4550 para
 * comunicación asíncrona a UART_BAUDRATE baudios, 8 bits de
 * datos, sin paridad, 1 bit de stop (8N1).
 * ----- */
void uart_init(void)
{
    /* Configuración de pines del módulo EUSART */
    TRISCbits.TRISC6 = 1; // TX (RC6) como entrada; el módulo lo controla
    automáticamente
    TRISCbits.TRISC7 = 1; // RX (RC7) como entrada

    /* Cálculo del registro de baud rate:
     * Usando BRG de alta velocidad (BRGH = 1, BRG16 = 0):
     * SPBRG = (Fosc / (16 * baudrate)) - 1
     */
    TXSTAbits.BRGH = 1; // Alta velocidad
    BAUDCONbits.BRG16 = 0; // Generador de 8 bits

    SPBRG = (unsigned char)((((_XTAL_FREQ / UART_BAUDRATE) / 16) - 1);

    TXSTAbits.SYNC = 0; // Modo asíncrono
    RCSTAbits.SPEN = 1; // Habilita el puerto serial (TX/RX)

    TXSTAbits.TX9 = 0; // Transmisión de 8 bits (sin noveno bit)
    RCSTAbits.RX9 = 0; // Recepción de 8 bits

    TXSTAbits.TXEN = 1; // Habilita transmisor
    RCSTAbits.CREN = 1; // Habilita receptor (modo continuo)

    /* Configuración de interrupciones de recepción */
    PIE1bits.RCIE = 1; // Habilita interrupción por recepción (RCIF)
    IPR1bits.RCIP = 1; // Prioridad alta para la interrupción de recepción

    RCONbits.IPEN = 1; // Habilita sistema de prioridades de interrupción

```

```

    INTCONbits.GIEH = 1;    // Habilita interrupciones de alta prioridad
    INTCONbits.GIEL = 1;    // Habilita interrupciones de baja prioridad

    /* Inicializa buffer circular vacío */
    rx_head = 0;
    rx_tail = 0;
}

/* -----
 * uart_write()
 * Transmisión por consulta (polling): espera a que el bit
 * TXIF indique que el registro TXREG está libre, y entonces
 * coloca el byte a enviar.
 * ----- */
void uart_write(unsigned char dato)
{
    while (PIR1bits.TXIF == 0)
        ;           // Espera a que el buffer de transmisión esté libre

    TXREG = dato;    // Carga el byte; el hardware lo transmite automáticamente
}

/* -----
 * uart_print()
 * Envía una cadena completa terminada en '\0', byte por byte.
 * ----- */
void uart_print(const char *cadena)
{
    while (*cadena != '\0')
    {
        uart_write((unsigned char)*cadena);
        cadena++;
    }
}

/* -----
 * uart_println()
 * Igual que uart_print(), pero agrega salto de línea al final.
 * Útil para imprimir cada lectura en una línea nueva del
 * monitor serial.
 * ----- */
void uart_println(const char *cadena)
{
    uart_print(cadena);
    uart_write('\r');
    uart_write('\n');
}

/* -----
 * uart_print_int()
 * Convierte un entero con signo a texto decimal y lo envía
 * por UART. Útil para monitoreo de variables/sensores.
 * ----- */
void uart_print_int(int numero)

```

```

{
    char buffer[7];          // suficiente para -32768..32767 + '\0'
    unsigned char i = 0;
    unsigned char j;
    unsigned char negativo = 0;
    unsigned int valor;

    if (numero < 0)
    {
        negativo = 1;
        valor = (unsigned int)(-numero);
    }
    else
    {
        valor = (unsigned int)numero;
    }

    /* Genera los dígitos en orden inverso */
    do
    {
        buffer[i++] = (valor % 10) + '0';
        valor /= 10;
    } while (valor > 0);

    if (negativo)
        buffer[i++] = '-';

    /* Envía los dígitos en orden correcto (invierte el buffer) */
    for (j = i; j > 0; j--)
        uart_write(buffer[j - 1]);
}

/* -----
 * uart_available()
 * Regresa 1 si hay al menos un byte sin leer en el buffer
 * circular de recepción, 0 si está vacío.
 * ----- */
unsigned char uart_available(void)
{
    return (rx_head != rx_tail);
}

/* -----
 * uart_read()
 * Extrae un byte del buffer circular (el más antiguo) y
 * avanza la cola. Si no hay datos disponibles regresa 0.
 * ----- */
unsigned char uart_read(void)
{
    unsigned char dato;

    if (rx_head == rx_tail)
        return 0;    // Buffer vacío, no hay nada que leer

    dato = rx_buffer[rx_tail];

```

```

        rx_tail = (rx_tail + 1) % UART_RX_BUFFER_SIZE;

    return dato;
}

/* -----
 * uart_read_line()
 * Copia caracteres del buffer circular hacia "destino" hasta
 * encontrar '\n', hasta llenar "tam_max - 1" caracteres, o
 * hasta vaciar el buffer disponible.
 *
 * Si se encontró '\n' regresa 1 y "destino" queda como cadena
 * terminada en '\0' (sin el '\n'). Si no se encontró salto de
 * línea (el comando aún no ha llegado completo), regresa 0 y
 * "destino" no debe usarse todavía.
 * ----- */
unsigned char uart_read_line(char *destino, unsigned char tam_max)
{
    unsigned char temp_tail = rx_tail;
    unsigned char i = 0;
    unsigned char encontrado = 0;

    /* Primero busca si ya existe un '\n' completo en el buffer,
     * sin retirar los datos todavía (solo explora) */
    while (temp_tail != rx_head)
    {
        if (rx_buffer[temp_tail] == '\n')
        {
            encontrado = 1;
            break;
        }
        temp_tail = (temp_tail + 1) % UART_RX_BUFFER_SIZE;
    }

    if (!encontrado)
        return 0;    // Todavía no ha llegado una línea completa

    /* Ahora sí copia y retira los bytes reales del buffer */
    while (uart_available() && i < (tam_max - 1))
    {
        unsigned char c = uart_read();

        if (c == '\n')
            break;

        if (c != '\r')    // ignora retorno de carro
            destino[i++] = c;
    }

    destino[i] = '\0';
    return 1;
}

/* -----
 * uart_flush()

```

```

    * Descarta cualquier dato pendiente en el buffer circular.
    * ----- */
void uart_flush(void)
{
    rx_head = 0;
    rx_tail = 0;
}

/* -----
 * uart_isr()
 * Rutina a llamar desde la interrupción de alta prioridad.
 * Cada vez que llega un byte nuevo (RCIF = 1), lo guarda en
 * el buffer circular. Si el buffer está lleno, descarta el
 * byte más antiguo (sobrescribe) para no bloquear la recepción.
 *
 * También limpia errores de overrun (OERR), comunes si el
 * programa principal tarda en leer el buffer.
 * ----- */
void uart_isr(void)
{
    unsigned char dato;
    unsigned char siguiente_head;

    if (PIR1bits.RCIF)
    {
        dato = RCREG;    // Leer RCREG limpia automáticamente RCIF

        siguiente_head = (rx_head + 1) % UART_RX_BUFFER_SIZE;

        if (siguiente_head != rx_tail)
        {
            /* Hay espacio: guarda el byte normalmente */
            rx_buffer[rx_head] = dato;
            rx_head = siguiente_head;
        }
        /* Si no hay espacio, se descarta el byte entrante
         * para no corromper el buffer (el más antiguo se
         * conserva hasta que el programa lo lea). */
    }

    /* Manejo de error de sobre-escritura (overrun).
    * Ocurre si llegan 3 bytes sin leer RCREG. Se limpia
    * apagando y encendiendo CREN. */
    if (RCSTAbits.OERR)
    {
        RCSTAbits.CREN = 0;
        RCSTAbits.CREN = 1;
    }
}

```

### 4.3 main.c — Ejemplo de uso

```

/* =====
 * main.c
 * Ejemplo de uso de la librería uart.h / uart.c
 * Tarjeta: Pinguino (PIC18F2550 / 4550)
 *
 * Funcionalidad de prueba:
 * - MONITOREO: cada cierto tiempo envía por UART el valor
 *   de un contador (simulando una lectura de sensor).
 * - CARGA: si llega una línea de texto desde la PC
 *   (terminada en Enter), la interpreta como comando simple:
 *   "LED ON" -> enciende un LED
 *   "LED OFF" -> apaga un LED
 *   cualquier otro texto -> lo regresa como eco
 *
 * Configuración de la terminal serial (ej. Tera Term, PuTTY,
 * el monitor del IDE Pinguino, etc.):
 * 9600 baudios, 8 bits de datos, sin paridad, 1 bit de stop
 * ===== */

#include "uart.h"

#define LED_PIN_BIT LATCbits.LATC0 // LED de prueba en RC0

/* -----
 * Vector de interrupción de alta prioridad (PIC18F).
 * Aquí se llama a la rutina de la librería UART para que
 * procese cada byte recibido sin bloquear el programa
 * principal.
 * ----- */
#pragma interrupt high_isr
void high_isr(void)
{
    uart_isr();
    /* Si tuvieras más periféricos con interrupción de alta
     * prioridad (Timers, ADC, etc.) sus rutinas también
     * se atenderían aquí. */
}

#pragma code high_vector = 0x08
void high_interrupt(void)
{
    _asm GOTO high_isr _endasm
}
#pragma code

/* Retardo simple por software */
void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    for (i = 0; i < ms; i++)
        for (j = 0; j < 800; j++)
            ;
}

/* Compara dos cadenas de texto (regresa 1 si son iguales) */

```

```

unsigned char str_equal(const char *a, const char *b)
{
    while (*a && *b)
    {
        if (*a != *b)
            return 0;
        a++;
        b++;
    }
    return (*a == '\0' && *b == '\0');
}

void main(void)
{
    char comando[32];
    unsigned int contador = 0;
    unsigned int tick = 0;

    TRISCbits.TRISC0 = 0;    // LED de prueba como salida
    LED_PIN_BIT = 0;

    uart_init();

    uart_println("== Pinguino UART listo ==");
    uart_println("Comandos: LED ON / LED OFF");

    while (1)
    {
        /* ----- CARGA: recepción de comandos ----- */
        if (uart_read_line(comando, sizeof(comando)))
        {
            if (str_equal(comando, "LED ON"))
            {
                LED_PIN_BIT = 1;
                uart_println("OK: LED encendido");
            }
            else if (str_equal(comando, "LED OFF"))
            {
                LED_PIN_BIT = 0;
                uart_println("OK: LED apagado");
            }
            else
            {
                uart_print("Comando no reconocido: ");
                uart_println(comando);
            }
        }

        /* ----- MONITOREO: envío periódico de datos ----- */
        delay_ms(10);
        tick++;

        if (tick >= 100)    // aprox. cada 1 segundo
        {
            tick = 0;
        }
    }
}

```

```

        contador++;

        uart_print("Contador = ");
        uart_print_int((int)contador);
        uart_println("");
    }
}

```

## 5. Notas de compilación y uso

---

- El reloj del sistema se definió en `_XTAL_FREQ = 48 000 000` Hz. Si tu configuración de fusibles usa otra frecuencia de oscilador, ajusta esa constante en `uart.h` para que el cálculo de `SPBRG` siga siendo correcto.
- El cálculo de `SPBRG` usa el modo de alta velocidad (`BRGH = 1`, `BRG16 = 0`), que da buena precisión a 9600 baudios con osciladores típicos de 48 MHz / 20 MHz / 8 MHz.
- `uart_isr()` debe llamarse desde la interrupción de **alta prioridad** (`high_isr`), como se muestra en `main.c`. El proyecto debe tener habilitado el sistema de prioridades (`RCONbits.IPEN = 1`), lo cual ya hace `uart_init()`.
- Para pruebas, cualquier programa de terminal serial (PuTTY, Tera Term, el monitor serial del IDE Pinguino, CoolTerm, etc.) sirve, configurado a **9600, 8N1**.
- `uart_read_line()` ignora el carácter `'\r'` y usa `'\n'` como marca de fin de línea — la mayoría de terminales seriales envían `\r\n` al presionar Enter, por lo que esto funciona sin configuración adicional.